UNIVERSIDAD DEL MAGDALENA · FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA EN CIENCIA DE DATOS · TERCER CUATRIMESTRE
CÁTEDRA DE PROGRAMACIÓN ORIENTADA A OBJETOS (C# / .NET 10)

# NexoLog S.A.S. — Guía de Arquitectura y Decisiones de Diseño

Aterrizando el proyecto: de las dudas iniciales al modelo de objetos limpio, defendible y funcional

**Docente Titular:** Ing. Andrés Eduardo Paternina Ariza Esp.

**Estado del Software:** Compilación con 0 errores y 101 pruebas superadas

### ■ Propósito de este documento para el equipo

Esta guía fue escrita para que **todo el equipo esté exactamente en la misma página**. Si sientes que el proyecto está 'en el aire', si aún crees que apenas estamos dibujando en Draw.io o si no has tenido tiempo de leerte todo el enunciado, este documento te explicará en español directo qué problema resolvemos, qué decisiones tomamos, por qué descartamos clases raras y cómo encaja cada pieza del código C# sin enredos.

## Bloque 1: Aterrizando el Problema Real (¿Qué nos pide el enunciado?)

NexoLog S.A.S. no es una tienda ni una fábrica: **es una empresa intermediaria logística**. Su negocio consiste en coordinar envíos entre establecimientos comerciales que quieren vender (Comercio Caribe, Comercio Andino, Comercio Sierra) y las empresas transportadoras que llevan los paquetes físicos a las ciudades de Colombia.

El gran reto del proyecto no es calcular una suma; el verdadero reto es que **la información nos llega dispersa en cinco archivos heterogéneos y con 'trampas' deliberadas** preparadas por el docente para poner a prueba si nuestro diseño es frágil o si es verdaderamente resiliente:

| Archivo Fuente | Formato | Contenido y Rol en el Sistema |
|---|---|---|
| **pedidos_caribe.json** | JSON | Solicitudes de Comercio Caribe (COM-01) y Comercio Sierra (COM-03). Contiene paquetes con pesos, distancias y destinatarios. |
| **pedidos_andino.xml** | XML | Solicitudes de Comercio Andino (COM-02). Mismo propósito que el anterior, pero estructurado en etiquetas XML. |
| **acuerdos.json** | JSON | Condiciones económicas pactadas con cada comercio: Tarifa por Peso ($3.500/kg), Tarifa por Distancia o Tarifa Fija. |
| **seguimiento_norte.json** | JSON | Acontecimientos de tracking (recepción, salida a ruta, entrega, devolución) para guías atendidas por Transportadora Norte. |
| **seguimiento_andina.xml** | XML | Acontecimientos de tracking para guías de Transportadora Andina, estructurados en formato XML. |

**■■ Las 4 trampas deliberadas de los archivos que nuestro código debió resolver**

1. **Eventos en desorden temporal:** En los archivos de tracking, un evento de 'Entrega' puede aparecer físicamente antes que la 'Recepción'. Si el sistema los procesara en el orden en que los lee, rechazaría la entrega por ilógica. La solución fue ordenar siempre por *FechaHora* ascendente.
2. **Datos inválidos y corruptos:** Paquetes con peso menor o igual a 0 kg, valores declarados negativos o comercios inexistentes. El sistema **no debe caerse** (no hacer crash con excepciones no controladas); debe registrar un RegistroInconsistencia y continuar procesando lo sano.
3. **Duplicados y conflictos:** Guías idénticas que llegan repetidas (se omite el duplicado en silencio) vs. guías con el mismo número pero distinta ciudad o destinatario (conflicto grave de auditoría).
4. **Cálculo condicionado de cobros:** Solo se cobra si el despacho terminó con estado **Entregado**. Si quedó Pendiente o Devuelto, el cobro es estrictamente $0.

## Bloque 2: Las 5 Decisiones de Diseño (Analizando las Dudas Iniciales)

Al iniciar el diseño, surgieron 5 observaciones comprensibles sobre el diagrama UML y la estructura de clases. A continuación analizamos con total transparencia técnica qué se decidió en cada caso y por qué:

### 1. ¿Hacían falta 3 Factorías (ModalidadFactory, TarifaFactory, AcontecimientoFactory)?

**Decisión tomada: RECHAZADO.** El PDF del proyecto solicita aplicar el *Patrón Factory* (en singular), no inventar tres factorías vacías. En nuestro sistema ya implementamos CreadorLectores.cs, que es una factoría real que evalúa si un archivo es .json o .xml y te entrega el lector adecuado desacoplando la serialización. Crear factorías solo para hacer new TarifaPorPeso() cuando el DTO ya conoce sus datos es sobreingeniería pura. Además, la diapositiva 14 de la Sesión 15 dicta una regla de oro: *'Trabajen sobre el código tal como está: no agreguen clases ni métodos que no existan'*.

### 2. ¿Debíamos crear una clase intermedia llamada 'AcuerdoComercial'?

**Decisión tomada: RECHAZADO.** En nuestro diseño, la clase Establecimiento tiene una relación directa con ITarifa (mediante el campo _tarifaAcordada). Introducir una clase AcuerdoComercial que solo sirva para guardar la tarifa sin tener ningún método ni cálculo propio es el antipatrón *Middle Man* (una clase cascarón que añade flechas al diagrama sin resolver nada). Establecimiento teniendo directamente su tarifa cumple limpiamente con el patrón Strategy.

### 3. ¿Acontecimiento debía tener 4 clases hijas (Recepcion, Salida, Entrega, Devolucion)?

**Decisión tomada: RECHAZADO.** Esta es una confusión común en POO: confundir un **registro de datos inmutable (Event Log)** con una jerarquía de comportamiento. Los 4 acontecimientos tienen exactamente los mismos datos: un ID, una guía, una fecha y un código (REC, RUT, ENT, DEV). Ninguno tiene un cálculo o algoritmo diferente al otro. La verdadera inteligencia y las reglas de negocio viven dentro de Despacho.ProcesarAcontecimiento(evt), que gobierna la máquina de estados. El polimorfismo del proyecto ya está sobradamente demostrado en las tarifas (ITarifa) y en las modalidades de SLA (ModalidadServicio).

### 4. ¿Debíamos mostrar los Lectores y Exportadores en el diagrama UML?

**Decisión tomada: ACEPTADO.** En el código C# sí existen las interfaces ILectorArchivo<T> e IExportadorInforme, la factoría CreadorLectores y las clases ExportadorJson y ExportadorXml. Mostrarlas en un paquete ordenado en el diagrama Draw.io le demuestra visualmente al profesor que cumplimos con los requisitos formales de Factory y Serialización JSON/XML exigidos en la rúbrica.

### 5. ¿Por qué en la cátedra de POO NO utilizamos rombos en los diagramas UML?

**Decisión tomada: REGLA DE CÁTEDRA CONFIRMADA.** Al revisar las diapositivas de clase del profesor Andrés Paternina (especialmente la Sesión 15, diapositivas 4, 6 y 7), se observa claramente que **el docente no utiliza rombos de agregación ni de composición**. El profesor modela las relaciones estructurales exclusivamente mediante **líneas simples de asociación con sus multiplicidades explícitas** (ej. 1 a 0..*), herencia con flecha de triángulo cerrado y realización de interfaz con línea discontinua. Llenar el diagrama de rombos no suma puntos en esta cátedra y solo causa confusiones de dirección. Líneas simples y multiplicidades claras son la norma.

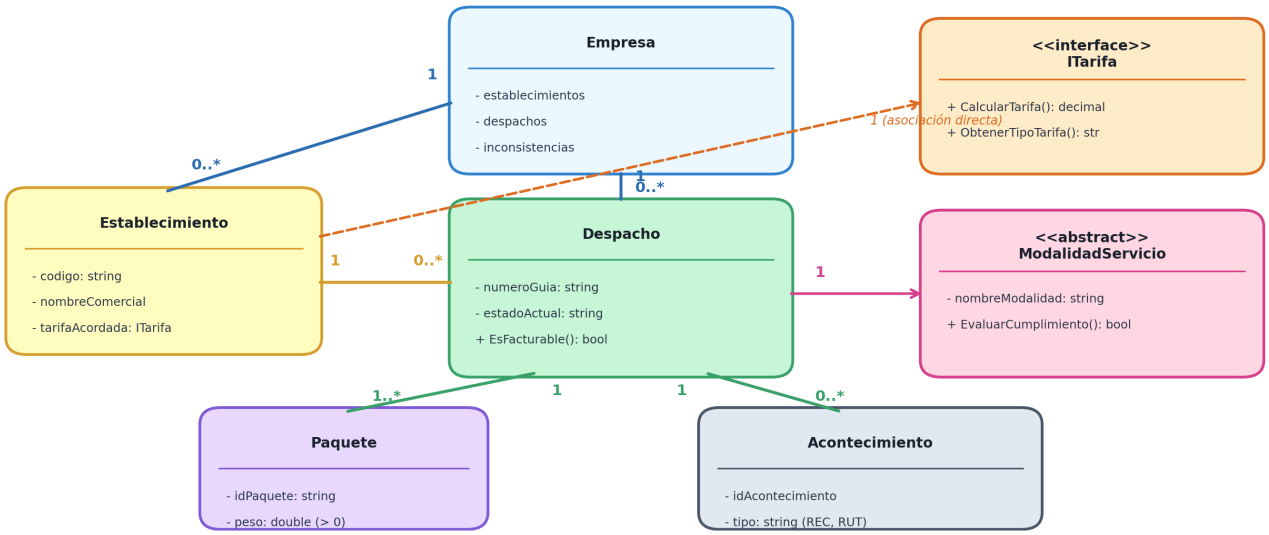



*Figura 1: Modelo de Clases de Dominio limpio, con asociaciones directas y multiplicidades según la norma de la Sesión 15.*

## Bloque 3: Anatomía de la Solución Limpia (Cómo encaja el código C#)

Muchos compañeros piensan que para programar hay que acumular cientos de líneas en una sola clase gigante. En nuestro proyecto hicimos exactamente lo contrario: **desacoplamos responsabilidades** para que cada clase sea corta, clara y fácil de entender.

### A. ¿Por qué NO borramos la clase 'Empresa', sino que la 'desintoxicamos'?

En un momento pensamos en eliminar Empresa.cs porque parecía un 'cajón de sastre' con casi 600 líneas que hacía de todo: leía archivos del disco, mapeaba JSONs y calculaba los reportes. Sin embargo, borrarla hubiera sido un grave error conceptual:

• En la diapositiva 7 de la Sesión 15, el propio profesor modela la clase Empresa como la raíz que contiene a los empleados.

• En el taller de alquiler de vehículos de la Sesión 16, Manuel y Jhon programaron una clase Empresa que administraba la flota de vehículos.
• Si la borrábamos, ¿dónde quedaban los despachos y los establecimientos en memoria? Hubieran terminado como variables sueltas en Program.cs, convirtiendo el programa en código procedimental plano sin objetos.

**La solución de ingeniería limpia (Opción A):** Mantuvimos Empresa como entidad de dominio puro, pero le quitamos el relleno ajeno. Pasó de 576 líneas a unas 80-100 líneas limpias. Ahora solo sabe registrar establecimientos, registrar despachos, ordenar seguimientos y delegar reportes.

## B. El concepto de Mappers y DTOs (Sesión 16 - Serialización)

Un **DTO (Data Transfer Object)** es un objeto plano que sirve únicamente para recibir los datos crudos del archivo JSON o XML (con nombres como numeroGuia, tarifaBase, etc.). En la Sesión 16 aprendimos que la mejor práctica es que el propio DTO sepa cómo transformarse en un objeto del dominio de negocio. Por eso creamos métodos como MapToDomain() en PedidoJsonDto y AcuerdoDto. Así, la capa de lectura se encarga del archivo, el DTO se encarga del mapeo y las clases de negocio (Despacho, Paquete) se mantienen 100% puras sin saber si vinieron de un JSON o de un XML.

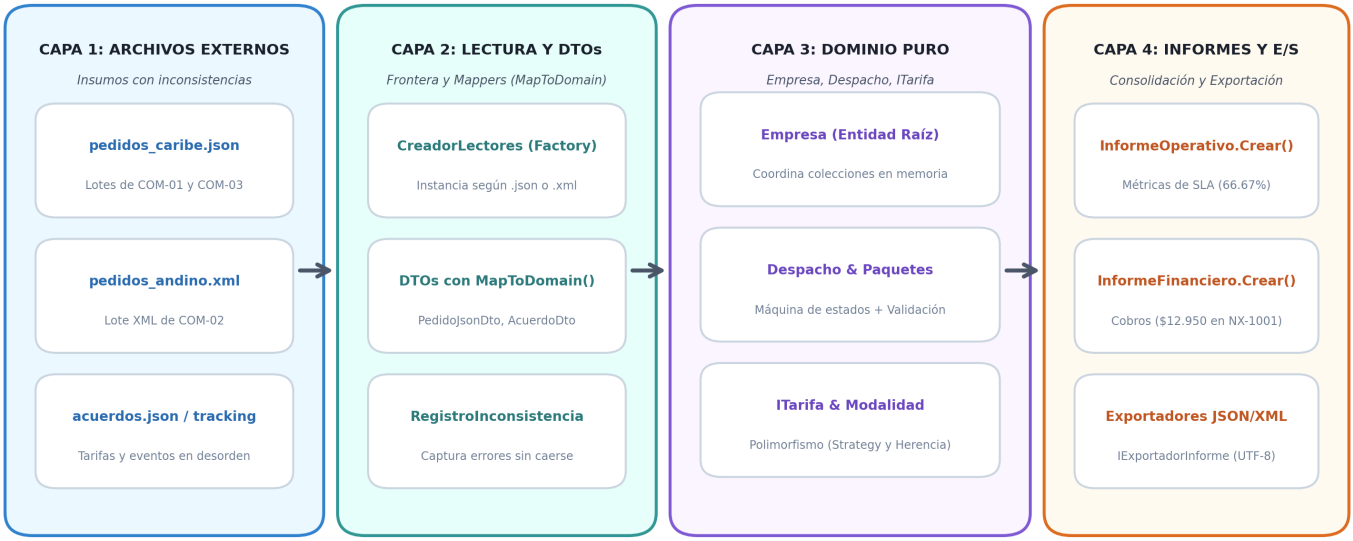



*Figura 2: Arquitectura en 4 Capas de NexoLog. Los datos viajan desde la izquierda (archivos) hacia el centro (dominio) de forma limpia.*

## C. ¿Por qué todas las clases tienen un constructor privado vacío?

Esta fue una de las mejores exigencias de la cátedra que habíamos olvidado: **siempre declarar un constructor vacío privado** (private MiClase() { }). ¿Por qué?

### ■■ Fundamento del Constructor Privado sin Parámetros

1. **Para Serialización y Reflexión:** Las librerías de .NET (System.Text.Json, serializadores XML) necesitan instanciar la clase sin argumentos antes de inyectarle los datos leídos del disco.
2. **Para Proteger el Encapsulamiento:** Si ese constructor fuera público (public Despacho()), cualquier programador descuidado podría hacer new Despacho() y dejar el objeto en un estado nulo, con guía vacía y paquetes rotos. Al marcarlo como private, obligamos a que en el código de negocio solo se pueda usar el constructor público que valida los datos con reglas estrictas (pesos > 0, cadenas no vacías).

## D. El ciclo de vida de un Despacho y la máquina de estados

Un despacho en NexoLog nace en el constructor siempre en estado Pendiente (nunca nulo). Conforme llegan los acontecimientos de tracking, cambia de estado siguiendo un orden lógico estricto:

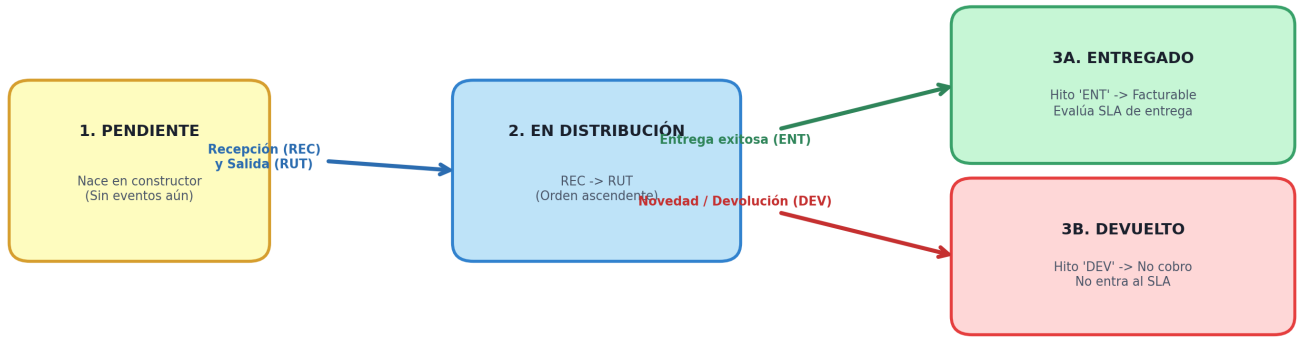



*Figura 3: Máquina de estados gobernada en Despacho.cs. Solo el estado Entregado habilita el cobro de la tarifa.*

# Bloque 4: Reglas de Oro de Cátedra y Resumen para el Equipo

Para que cualquier integrante del equipo pueda explicar con soltura cómo está construido el proyecto, aquí está la síntesis de las reglas de oro que se respetaron al 100% en todo el repositorio:

| Regla de Oro | Restricción | Cómo lo resolvimos en NexoLog |
|---|---|---|
| **Cero 'readonly'** | Prohibido en la cátedra. | Toda protección de solo lectura se hace con campos privados convencionales y propiedades con solo 'get'. |
| **Cero 'enum'** | No usamos enums. | Estados como 'Pendiente', 'Entregado' o 'REC' se modelan como constantes 'public const string'. |
| **Declaraciones individuales** | Una línea por variable. | Prohibido 'double a, b;'. Cada variable tiene su tipo explícito en su propia línea independiente. |
| **Auto-propiedades limpias** | Mínima redundancia. | Si una propiedad no valida nada en el setter, usamos auto-propiedad compacta con 'private set'. |
| **Cero 'switch'** | Usar if / else if / else. | Todas las decisiones de control de flujo se implementan con estructuras condicionales tradicionales. |

| | | |
|---|---|---|
| **Codificación UTF-8** | Obligatoria en I/O. | Toda lectura o escritura de archivos especifica explícitamente 'Encoding.UTF8' para evitar caracteres corruptos. |
| **Cero rombos en UML** | Convención de cátedra. | En los diagramas de Draw.io usamos únicamente líneas directas de asociación con sus multiplicidades. |

### ■ Las 3 cifras clave del negocio que validan que el sistema funciona perfecto

• **Guía de Control NX-1001:** Pesa exactamente 3.7 kg y pertenece a Comercio Caribe (COM-01), que cobra tarifa por peso ($3.500/kg). Cobro exacto: 3.7 × 3.500 = **$12.950**.
• **Cumplimiento de SLA (66.67%):** De 11 despachos totales, 6 fueron entregados. De esos 6, exactamente 4 cumplieron los tiempos de su modalidad (estándar, exprés o programado) y 2 no. Porcentaje: (4 / 6) × 100 = **66.67%**. Los pendientes y devueltos no entran al denominador.
• **Facturación Total: $104.450** cobrados limpiamente sobre los despachos entregados.

### ■ Mensaje final para el equipo: Tranquilidad total

Compañeros: **el proyecto está completamente construido, compila con 0 errores y supera 101 pruebas automatizadas**. No necesitan estresarse pensando que 'no hemos empezado' o que estamos atrasados. Todo el trabajo duro de arquitectura, desacoplamiento y resiliencia ya está hecho y comprobado. Lo único que necesitamos es leer esta guía con calma, entender cómo viajan los datos y tener la tranquilidad de que nuestra solución es simple, elegante, limpia y 100% defendible.